# Deximon — Project Proposal (EECS4314)

**Team:** Deximon (5 members) **Course:** EECS4314 – Advanced Software Engineering, Winter 2026
**Submission Date:** 2026-01-27 **Status:** Draft — pending TA sign-off

Related:

- [[project-proposal|Course proposal template & instructions]]
- [[detailed-design|Course detailed-design template]] (due 2026-02-13)
- [[pokevault-analysis|Pre-decision analysis of the original PokéVault idea]]

---

## Q1 — Product Name

**Deximon**

---

## Q2 — Core Concept

Deximon is a social, web-based platform for Pokémon trading-card collectors. It combines three pillars: a personal digital binder, a card-recognition scanner, and a peer-to-peer trade/sale marketplace.

**User functionality:**

- **Account & profile.** Users sign up, customize a public profile, and visit other collectors' profiles. Binders are public by default with an opt-in privacy toggle.
- **Digital binder.** Each user maintains a 9-slot-per-page binder of the cards they own. Cards are presented with art and rarity-appropriate visual effects — holo and reverse-holo cards display interactive, mouse-reactive shine animations that mimic real foil. Users drag cards between slots and pages to organize their binder.
- **Add cards by scanning.** Users upload a photo of a physical card. The system runs the image through a card-recognition pipeline (image preprocessing → AWS Rekognition text detection → fuzzy match against the Pokémon TCG API catalog) and pre-fills the card's identifying fields. The user then completes attributes the scanner cannot reliably determine (holo / reverse-holo, condition, language, etc.) and saves the card to their binder. A fully manual add path is also available.
- **Marketplace.** A dedicated Marketplace tab lets users search and filter cards that other users have listed for trade or sale. Filters include name, set, rarity, type, condition, and listing state. Each listing is tied to a specific card in a seller's binder; clicking a listing reveals card details, the seller's profile, and an "Open Chat" action.
- **Listings & chat.** Cards in a user's binder can be flipped to one of four listing states — *Available*, *On Hold*, *Sold*, *Cancelled* — with an optional asking price. Buyers can only start a chat thread by acting on a specific listing, which scopes the conversation to that card and reduces spam. Trade or payment logistics are arranged between users; Deximon does **not** process payments.
- **Notifications.** In-app, realtime notifications for new messages and listing state changes.

---

## Q3 — Team Members

1. Ege Yesilyurt
2. Brian Chang-Kit
3. Abhiroop Sikand
4. Neel Upadhyay

5. Weiqin Situ

*Note: Team size is 5. The course rubric specifies teams of 6–7; we will confirm acceptance of a 5-person team with the TA at sign-off, or accept a merge with another partial team.*

---

## Q4 — Programming Languages & Web Frameworks

**Frontend**

- TypeScript, React, **Next.js** (App Router, server-side rendering for public profiles)
- Tailwind CSS for styling; CSS gradients and mouse-tracked transforms for holo animations
- `dnd-kit` for drag-and-drop binder interactions
- Native browser **WebSocket** client for realtime chat and notifications

**Backend (primary)**

- **Python 3.12** with **FastAPI** for the main REST API: auth, user profiles, binders, listings, search, and chat persistence
- **SQLAlchemy 2 + Alembic** for Postgres persistence and migrations
- **Pydantic v2** for request/response schemas and validation
- **JWT-based authentication** via `python-jose` + `passlib[bcrypt]` (email/password + OAuth via Google through `Authlib`)
- **FastAPI native WebSocket** endpoints for realtime chat threads, presence, and notifications
- **Uvicorn** as the ASGI server; **Poetry** for dependency management

**Backend (card-recognition microservice)**

- **Python 3.12** with **FastAPI**
- **Pillow / OpenCV** for image preprocessing
- **AWS Rekognition** (`DetectText`) for OCR
- Fuzzy text matching (e.g. `rapidfuzz`) against the **Pokémon TCG API** card catalog
- Deployed independently from the main backend (different resource profile, heavier image-processing dependencies)

**Data layer**

- **PostgreSQL** (AWS RDS) — primary relational store for users, cards, listings, messages, binder layouts
- **AWS S3** — uploaded card photos and profile assets

**Infrastructure & tooling**

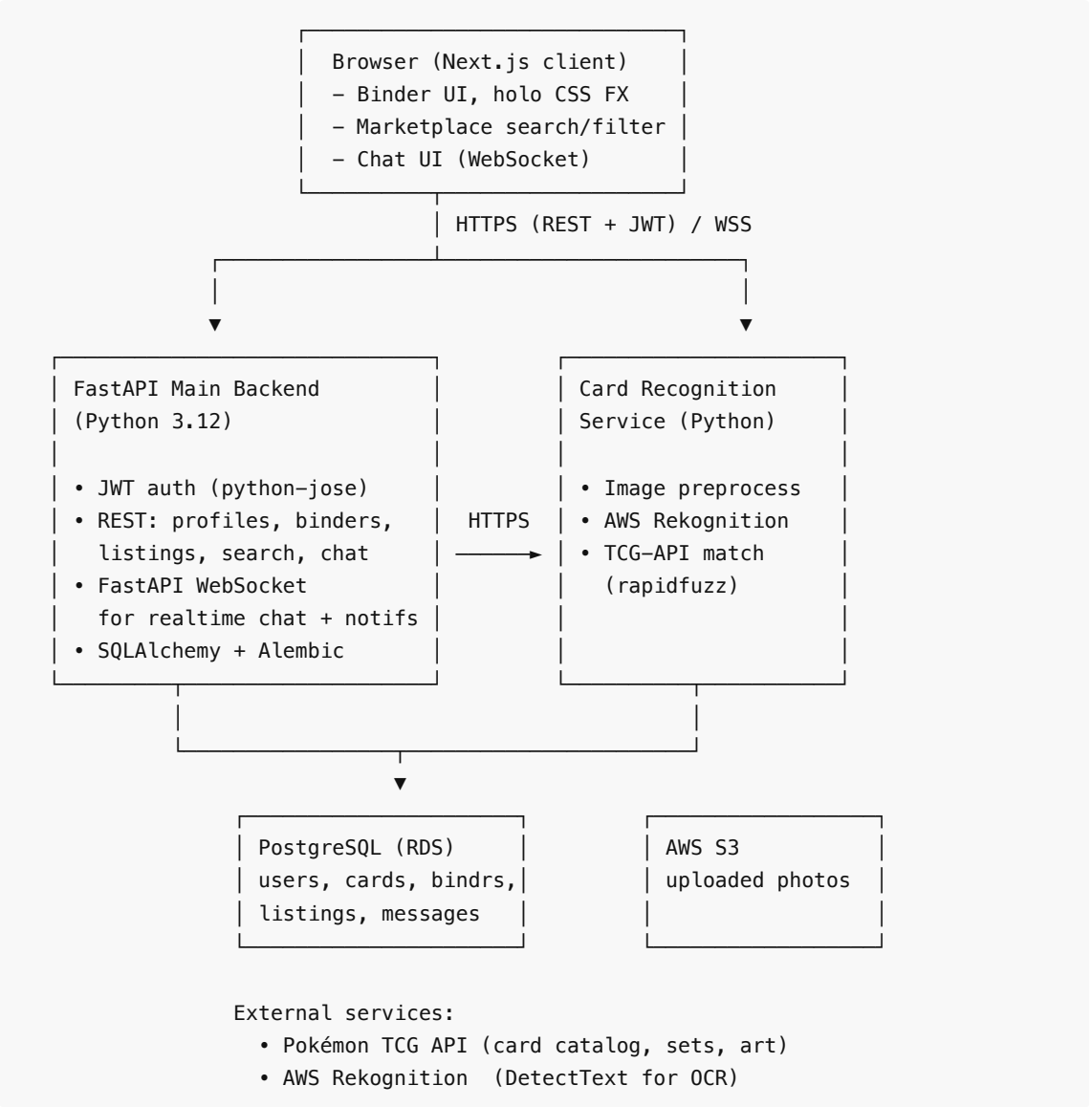
- **AWS** (EC2 / ECS for services, RDS for Postgres, S3 for object storage, Rekognition for OCR, SES later as a stretch goal for email)
- **GitHub** for source control, with feature-branch + pull-request + code-review workflow
- **GitHub Actions** for CI (lint, type-check, unit tests, E2E smoke tests)
- **Docker** for service containerization
- **pytest** (both Python services), **Vitest** (TypeScript frontend), **Playwright** (end-to-end)

---

## Q5 — System Architecture, Requirements, and Learning Goals

### High-Level Architecture

Deximon is structured as three deployable services backed by a shared data layer and one external AI service.




### Data Flow — Representative Scenarios

**Scanning a card.** Client requests a pre-signed S3 URL from the FastAPI backend → uploads the image directly to S3 → calls the Card-Recognition Service with the S3 key → service preprocesses the image, calls Rekognition for raw text, fuzzy-matches against the TCG-API catalog, returns a structured card candidate → user confirms / corrects fields in the client → the main FastAPI backend writes the card to Postgres and updates the binder layout.

**Initiating a trade chat.** Buyer opens a listing on the Marketplace tab → clicks "Open Chat" → FastAPI creates (or fetches) a listing-scoped chat thread → client opens a WebSocket connection to the thread's endpoint → both users exchange messages in realtime, persisted to Postgres → listing state transitions (Available → On Hold → Sold) are broadcast on the same channel.

### Project Requirements We Strive to Implement

- **Authenticated multi-user system** with public profiles and per-user state

- **Persistent state** across all features (binders, listings, chat history) in PostgreSQL
- **Search and filter** over a large, externally-sourced dataset (the TCG card catalog)
- **External API integration** with the Pokémon TCG API
- **Cloud AI integration** with AWS Rekognition
- **Realtime functionality** via websockets (chat + notifications)
- **Multi-service architecture** — a Next.js client, a FastAPI main backend, and a separate Python card-recognition microservice, each independently deployable with distinct responsibilities and resource profiles
- **Inter-user interaction** — profiles, browsing others' binders, listing-scoped chat, a trade/sale marketplace
- **Production-style DevOps** — Docker, AWS deployment, CI/CD via GitHub Actions
- **Automated testing** at unit, integration, and end-to-end levels

### Learning Goals Fulfilled

- **Designing a non-trivial distributed system.** The architecture (Next.js client + FastAPI main backend + Python card-recognition microservice) forces explicit design decisions about service boundaries, inter-service contracts, asynchronous processing, and failure modes — exactly the architecture-level reasoning the course emphasizes.
- **Working in a real team workflow.** Feature branches, mandatory pull-request reviews, CI gates, and clear ownership of sub-systems mirror professional engineering practice.
- **Integrating external services responsibly.** Pokémon TCG API and AWS Rekognition each bring rate limits, latency, cost, and failure cases that must be handled (caching, retries, fallback paths).
- **Testing across layers.** Unit tests for business logic (e.g. listing state transitions), integration tests for service boundaries, and Playwright E2E tests for golden-path user flows (signup → scan → list → chat).
- **Deployment and operability.** Containerized services on AWS with managed Postgres and S3 expose the team to real cloud infrastructure concerns — provisioning, secrets management, environment parity, and cost control under a free-tier budget.

### Scope Discipline (Stretch Goals, Not MVP)

To keep scope realistic for one semester with five members, the following are explicitly marked as **nice-to-have** and will only be built after the core MVP is complete and stable:

- Email notifications via AWS SES
- WebGL / shader-based holo animations beyond what CSS can achieve
- Mobile-app wrapper
- Price-history tracking and alerts
- Trade-suggestion engine (wishlist ↔ duplicates matching)